

SIT307 Task 11.1HD — Video Presentation Script (≤ 5 minutes)

How to use this: do *not* read it word for word. Read it aloud twice, then record from the bullet of each segment in your own words; tutors can tell when someone is reading. Every number below was filled in automatically from **results/**, so rerun **python run_all.py** (with xgboost) before recording and the script updates to match your run. About 130-150 spoken words per minute; the SAY text totals roughly 700 words.

Before recording: camera on with your face visible throughout (small webcam overlay is fine); open in tabs: the report PDF, VS Code with **src/**, a terminal in the project folder, and the executed notebook. Increase editor/terminal font size so text is readable in the video. Brief order matters: present the **proposed solution first**, then the reproduction, then the code demo/results.

0:00 - 0:20 Introduction

ON SCREEN: Face camera, then the report title page.

SAY: Hi, I'm [name], and this is my SIT307 HD mini-research. I picked the paper by Bhagat and colleagues from 2025, which claims a stacking ensemble predicts heart disease with 98.5 percent accuracy. I reproduced the whole study, found out why that number is so high, and designed a method that fixes the problem. I'll start with my proposed solution, then show the reproduction, and finish with a live demo and results.

0:20 - 1:35 Part 2 — Proposed solution (LF-DPS)

ON SCREEN: Report Section 4.2 and Table 11 (paper vs proposed). Then switch to **src/part2_proposed.py**: scroll the docstring (C1-C5), then point at **typed_preprocessor()** in **common.py**, **tune_pool()** and **greedy_select()**.

SAY: The main limitation of the paper is its evaluation. The dataset has 1025 rows but only 302 unique patients — each patient is copied three or four times — and the paper splits rows at random, so copies of the same patient end up in both training and test. My method, which I call LF-DPS, Leakage-Free Diversity-Pruned Stacking, has five parts. One: I collapse duplicates so the unit is the patient, and evaluate with five-by-five repeated cross-validation. Two: I found that `ca` equal to 4 and `thal` equal to 0 aren't valid codes — they match exactly the missing values in the original UCI file — so I treat them as missing and impute inside each fold. Three: categorical features like chest-pain type are one-hot encoded instead of being treated as numbers, which matters for KNN and logistic regression. Four: every model is tuned with randomized search in an *inner* loop, so tuning never sees the test patients. And five, the actual ensemble change: instead of stacking all six models, a greedy search adds one model at a time only if it raises the meta-learner's cross-validated AUC. A model that's highly correlated with ones already chosen can't add AUC, so this naturally picks complementary models.

1:35 - 2:45 Part 1 — Reproduction and critical analysis

ON SCREEN: Terminal: show `python src/part1_reproduce.py` (start it, or show the saved `results/part1_log.txt`). Open report Table 6 (paper vs reproduced), then Table 3 (data audit), then Fig. 3 (leakage) and Fig. 4.

SAY: For the reproduction I used the same 13 features and six classifiers with 5-fold stacking. The paper doesn't report its split, seed, hyper-parameters or meta-learner, so I inferred them: 98.53 percent is exactly 202 out of 205, which means an 80/20 split. With seed 42, my stacking gets 98.54% with precision 1.00 — essentially the paper's result — and over 30 different splits it averages 99.7%. So it's reproducible. But when I audited the data, on average 97.1% of test rows had an identical copy in training. On those rows the tree models and stacking are about 100% correct; on the genuinely unseen ones they drop to between 86% and 93%. They're memorising, not generalising. When I rerun the paper's exact pipeline on unique patients, stacking falls to 81.9%, and the best model becomes LR. I also found errors in the paper's results table — for example MCC is reported as 1.0 alongside three false negatives; recomputed, it's about 0.97.

2:45 - 4:15 Demo and results of the proposed method

ON SCREEN: Notebook (already executed): show the Part 2 summary table cell, then the boxplot figure (Fig. 6), the stats table (Table 13), the selection-frequency/correlation figure (Fig. 8), and finally Fig. 9 (E6). Optionally run `python run_all.py --report` to show the PDF regenerating from results.

SAY: Here are the results on unique patients, with all methods on the same 25 folds. The ablation goes from the paper's stacking, A0, to my full method, A3. LF-DPS reaches 83.8% accuracy versus 82.4% for the paper's method, and higher F1 and MCC, using on average 2.1 models instead of six. But I want to be honest: with the Nadeau-Bengio corrected t-test, which accounts for overlapping training sets in repeated CV, that accuracy gain has p equal to 0.44 — not significant. With only 302 patients, about 61 per test fold, the fold-to-fold variation is bigger than the gain. The selection behaves as intended: the decision tree was never chosen, and random forest and boosting, which are correlated at 0.96, rarely appear together. The strongest evidence is this last experiment. I locked away 20 percent of patients that no protocol ever sees. The paper's protocol estimated 99.6% but got 80.7% on those new patients — over-estimating by 19 points. LF-DPS estimated 83.6% and achieved 83.6%. So my method's numbers can actually be trusted, and it's also more accurate on genuinely new patients.

4:15 - 4:50 Conclusion

ON SCREEN: Face camera (or report Section 5 / Table 15).

SAY: To sum up: the paper's 98.5 percent is reproducible, but it's an artefact of duplicate patients leaking between training and test. On real unseen patients, these 13 variables support roughly 82 to 84 percent. My method gives an honest estimate, handles the data errors properly and uses a smaller ensemble. The main limitation is that it's one small cohort; next I'd validate on the Hungarian and Swiss cohorts as a true external test, and add cost-sensitive thresholds, since missing a sick patient is worse than a false alarm. The lesson for me was that validation design mattered far more than which model you pick.

4:50 - 5:00 Close

ON SCREEN: Report title page showing the code link.

SAY: All code, data and the notebook are at the link in my report, and running run_all.py regenerates every number you've seen. Thanks for watching.

Note: this script was generated with the XGBoost stand-in. Rerun with xgboost installed before recording.

Viva / Q&A; crib sheet — understand these, don't memorise them

Why does a random row split leak here?

Each patient appears 3–4 times. A random split puts copies of the same patient on both sides, so the test set mostly contains patients the model was trained on. It measures recall of memorised rows, not generalisation.

Why do trees benefit more than logistic regression?

An unpruned tree or random forest can fit training rows perfectly (training accuracy ≈ 1.0), so an identical test row gets the right answer for free. Logistic regression has a single linear boundary and can't memorise individual points, so duplicates barely help it.

How did you infer the paper's split?

$0.9853 \times 205 = 202$ correct; 205 is 20% of 1,025. Precision 1.0 and recall 0.9727 imply TP=107, FN=3, FP=0, TN=95.

What is nested cross-validation and why use it?

The outer loop estimates performance; the inner loop (only on outer-training data) chooses hyper-parameters. If you tune and report on the same folds, you pick the configuration that got lucky on those folds, which biases the estimate upward (Cawley & Talbot 2010).

How does stacking work?

Base models produce out-of-fold predicted probabilities on the training set (via internal 5-fold CV, so a model never predicts a row it was trained on). A meta-learner — logistic regression — is trained on those probabilities to combine them.

How does your greedy selection reward diversity?

It starts with the model whose OOF probabilities give the best meta-learner AUC, then adds the model that increases AUC most, stopping when the gain is below 0.002. A model nearly identical to one already in the stack adds no new information, so it can't raise AUC and isn't chosen.

Why the corrected (Nadeau-Bengio) t-test?

In repeated CV the training sets overlap heavily, so fold scores aren't independent; a normal paired t-test underestimates variance and gives too-small p-values. The correction inflates the variance term by $(1/J + n_{\text{test}}/n_{\text{train}})$.

Your method isn't significantly better — so what did you improve?

The validity of the evaluation. E6 shows the paper's protocol over-estimates by about 19 points while mine is within less than one point of true external accuracy. Plus correct data handling, a smaller ensemble, and higher accuracy on genuinely new patients. Small data limits how much any model can gain.

Why is ca = 4 a missing value?

The paper's own table says ca is 0–3 and thal 1–3. In unique rows there are exactly 4 patients with ca=4 and 2 with thal=0 — the same as the 4 and 2 '?' values in the original UCI Cleveland file.

Why one-hot encode chest-pain type?

cp is nominal: 0–3 are categories, not amounts. Treated as a number, KNN thinks type 3 is 'three times' type 1, and logistic regression forces a single monotonic effect.

What is the Brier score?

Mean squared error between predicted probability and the 0/1 outcome; lower is better. It measures calibration and sharpness, which matters clinically when probabilities guide decisions.

What would you do next?

External validation on other hospitals' cohorts, cost-sensitive thresholds to favour recall, and reporting under the TRIPOD+AI guideline.

Did you use AI tools?

Answer honestly and consistently with your report's GenAI acknowledgement: say what you used it for and what you checked and decided yourself.

Recording checklist

- ☐ Face visible the whole time; audio clear (test 10 seconds first).
- ☐ Under 5:00 — anything after 5 minutes may not be marked.
- ☐ Proposed solution presented before the reproduction.
- ☐ Show real code running or real outputs, not only slides.
- ☐ Upload to YouTube as **Unlisted**; open the link in a private window to check it plays.
- ☐ Paste the link into the STUDENT block of make_report.py and rebuild the report.